Car Rental Fleet Manager — Part 3

Module 6: Collections & Streams
This part contains the main program. It creates the fleet, runs the search and sorting features, filters the actual fleet with a stream, and saves the fleet.

Code for this part

```
// CarRentalApp.java — PART 3 (Module 6)
// Save this file as CarRentalApp.java

import java.util.List;
import java.util.stream.Collectors;

public class CarRentalApp {
    public static void main(String[] args) {
        try {
            FleetManager fleet = new FleetManager();

            fleet.addCar(new Car("Honda City", 2200));
            fleet.addCar(new Car("Tata Nexon", 1800));
            fleet.addCar(new Car("Hyundai i20", 1500));

            System.out.println("ALL CARS");
            fleet.showCars();

            System.out.println("\\nSEARCH RESULTS");
            fleet.searchCar("tata");

            System.out.println("\\nSORTED FLEET");
            fleet.sortCars();
            fleet.showCars();

            // Stream example: find cars under Rs.2000 per day
            List<Car> budgetCars = fleet.getCars().stream()
                .filter(car -> car.getDailyRate() < 2000)
                .collect(Collectors.toList());

            System.out.println("\\nCARS UNDER Rs.2000");
            budgetCars.forEach(System.out::println);

            fleet.saveCars();

        } catch (IllegalArgumentException | IllegalStateException e) {
            System.out.println("Rental error: " + e.getMessage());
        }
    }
}
```

Save as: `CarRentalApp.java`. Put all three .java files in one folder. Compile with `javac Car.java FleetManager.java CarRentalApp.java` and run with `java CarRentalApp`.